

How to protect yourself from Bluetooth security risks

Backdoors and security flaws in Bluetooth devices can be exploited if not detected early. Conducting Bluetooth security audits is crucial to prevent potential threats.

- **Use security tools.** Many existing tools are outdated or require specialised hardware. BluetoothUSB by Tarlogic simplifies security audits across all devices, regardless of the operating system.
- **Keep track of your device.** Always know where your phone or laptop is, especially in public spaces. Set up a find my device service to remotely lock a lost phone.
- **Avoid Bluetooth for sensitive data.** Never transfer passwords or important files over Bluetooth unless encrypted first.
- **Turn off Bluetooth when not in use.** This minimises attack risks and saves battery life.

By following these steps, you can reduce vulnerabilities and improve Bluetooth security.

in Bluetooth communication and are typically used for internal testing. While debugging tools are standard in Bluetooth controllers, the presence of undocumented commands raises concerns about potential security risks and unauthorised access.

This vulnerability affects many ESP-based devices due to its origin in the Bluetooth HCI layer. While most ESP development boards and Bluetooth-enabled devices are at risk, some remain unaffected, like IndusBoard (which features an advanced ESP32 chip but does not use its Bluetooth function). Likewise, devices such as the IndusBoard ESP32-S2, which lacks Bluetooth, are secure.

That said, the actual impact on users remains unclear. One key question is how these commands are executed—whether they require a malicious firmware update to trigger them, in which case the firmware itself could pose a greater risk. Many microcontroller manufacturers, including TI and others, also include test commands, but they are typically documented. These undocumented ESP32 commands may serve a similar purpose. Given the widespread use of ESP32-based devices, further investigation will likely reveal more details soon.

Protecting against security risks

One of the first steps is to audit and remove hidden or debug BLE services. Firmware should be thoroughly

checked for test UUIDs, OTA triggers via BLE, and debugging services that should not be present in production. Using BLE scanner tools like nRF Connect or LightBlue can help verify what the device exposes. By removing unintended services, the attack surface can be significantly reduced.

If the device does not require BLE functionality in production, it is best to disable the BLE stack entirely. This can be done in the firmware using commands like `btStop()`; for Arduino or `esp_bt_controller_disable()`; for ESP-IDF. Disabling unused features eliminates unnecessary security risks.

Ensuring firmware security at the code level is also essential. Developers should avoid leaving hidden BLE services, test UUIDs, or OTA triggers exposed. Proper authentication mechanisms, such as pairing, bonding, and encrypted communication, should consistently be implemented. Additionally, default BLE pairing keys like 123456 or 000000 should never be used, as they are easily exploited. MAC address filtering (whitelisting or blacklisting) can also restrict access to authorised devices only.

Firmware updates (OTA) should be secured to prevent unauthorised modifications. Using ESP-IDF Secure Boot and Flash Encryption ensures that only signed firmware can run and that flash contents remain encrypted. OTA triggers should not be exposed via BLE without strong authentication,

and if OTA is not required, it should be disabled completely.

Regularly scanning for unintended services is another critical step in securing BLE devices. Tools like nRF Connect, LightBlue, and BLE Scanner Pro can help detect any unintended BLE services or test/debug endpoints that were not removed. Additional penetration testing can be performed using tools such as ``gatttool``, ``blejack``, and Wireshark with BLE sniffing to identify and fix security flaws.

Physical security measures should also be considered to prevent hardware-based attacks. Debug interfaces like JTAG and UART should be disabled on production boards. Using eFuses can help lock flash readouts and protect secure boot keys. Additionally, serial flashing mode should be disabled via boot pins after deployment to prevent unauthorised firmware modifications.

Discovering hidden functionalities in the ESP32 highlights potential security risks attackers could exploit to compromise IoT devices. While Espressif asserts that these commands are meant for debugging, their undocumented nature raises concerns about unauthorised access and spoofing attacks. Securing BLE-enabled devices requires a proactive approach, including auditing firmware, disabling unnecessary services, implementing strong authentication, and securing OTA updates. As research continues, manufacturers and developers must prioritise security measures to mitigate risks and protect sensitive systems from potential exploitation. **EFY**

Nidhi Agarwal is Senior Technology Journalist at EFY with a deep interest in embedded systems, development boards, and IoT cloud solutions.

Ashwini Kumar Sinha, an IoT and AI enthusiast, is Tech Journalist at EFY.